FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

MIDTERM

Calvin Xanneder Somera Sison	**Mr. Hadji Tejuco**
Name of Student	Name of Professor
October 8, 2026	**October 8, 2026**
Date Performed	Date Submitted

Program Development Steps

The program was developed as a 32-bit NASM application for Linux. It presents a digital slambook using organized terminal text, row and column positioning, and ASCII art.

1. I reviewed the required slambook fields and planned a terminal layout that would remain readable in a wide console window.
2. I gathered and shortened the personal information so each item would fit inside the 100-column design.
3. I created the NASM source file and declared the program as 32-bit code using BITS 32 and GLOBAL _start.
4. I stored the labels, personal information, borders, and CXS ASCII art in the data section.
5. I created the PRINT_AT macro to pass the requested row, column, message address, and message length to the positioning procedure.
6. I implemented the print_at procedure. It prints line breaks and spaces to move to the requested location, then uses the Linux sys_write service to display the selected text.
7. I arranged the program calls from top to bottom and from left to right to form the title, About Me section, My Firsts section, favorites, interests, and ASCII art.
8. I assembled the source using NASM with the ELF32 format and linked the object file as an i386 executable.
9. I ran the program in the Docker NASM environment, checked the alignment, and captured the final output for documentation.

Commands Used

```
nasm -f elf32 Calvin_Midterm_Slambook.asm -o Calvin_Midterm_Slambook.o
ld -m elf_i386 Calvin_Midterm_Slambook.o -o Calvin_Midterm_Slambook
./Calvin_Midterm_Slambook
```

Program Functionalities

32-bit program setup. BITS 32 selects 32-bit instructions, while GLOBAL _start exposes the program entry point to the linker.

PRINT_AT macro. This macro accepts a row, column, message label, and calculated message length. It makes the main program easy to read because every output item is assigned a clear screen position.

Position tracking. The current_row and current_col variables remember the program's latest printing location. This allows later messages to be placed with the correct number of line breaks and spaces.

print_at procedure. This procedure moves downward by printing newline characters, moves right by printing spaces, and then writes the requested message.

Linux output service. The program uses int 0x80 with EAX equal to 4 for sys_write, EBX equal to 1 for standard output, ECX as the message address, and EDX as the message length.

Organized slambook sections. The output separates personal information, memorable first experiences, favorites, and weekend interests using headings and horizontal ASCII borders.

ASCII art. A large CXS design is placed near the bottom as a personalized visual element required by the activity.

Program termination. The program uses the Linux sys_exit service with a zero exit status after the complete design is displayed.

Source Code

```
; ================================================================
; COMPUTER SYSTEMS AND ARCHITECTURE - MIDTERM PROJECT
; Digital About Me Slambook
; Student: Calvin Xanneder Somera Sison
; Student Number: 202511575
;
; Platform: NASM 32-bit Linux (ELF32)
; The PRINT_AT macro and print_at procedure arrange every item by
; row and column without using ANSI cursor-control sequences.
; ================================================================

BITS 32
GLOBAL _start

%macro PRINT_AT 3
    mov esi, %1              ; target row
    mov edi, %2              ; target column
    mov ecx, %3              ; address of text
    mov edx, %3_len          ; length of text
    call print_at
%endmacro

SECTION .data
    newline        db 10
    one_space      db ' '

    border         db '+------------------------------------------------------------------------------------------------------------+'
    border_len     equ $ - border

    title          db "CALVIN'S DIGITAL SLAMBOOK"
    title_len      equ $ - title
    subtitle       db 'COMPUTER SYSTEMS AND ARCHITECTURE MIDTERM PROJECT'
```

Source Code Continued

```
subtitle_len    equ $ - subtitle

about_header    db '[ ABOUT ME ]'
about_header_len equ $ - about_header
name_text       db 'Name: Calvin Xanneder Somera Sison'
name_text_len   equ $ - name_text
id_text         db 'Student No.: 202511575'
id_text_len     equ $ - id_text
email_text      db 'Email: cssison@fit.edu.ph'
email_text_len  equ $ - email_text
website_text    db 'Blog/Website: None'
website_text_len equ $ - website_text

firsts_header   db '[ MY FIRSTS ]'
firsts_header_len equ $ - firsts_header
achievement     db 'First big achievement: Ranked 1st in my Grade 5 class.'
achievement_len equ $ - achievement
risk_text       db 'First risk: Making new friends after transferring to a new school.'
risk_text_len   equ $ - risk_text
happy_line1     db 'Happiest moment: Both of my separated parents attended my Grade 6 graduation'
happy_line1_len equ $ - happy_line1
happy_line2     db '                together and joined me on stage.'
happy_line2_len equ $ - happy_line2

favorites_header db '[ MY FAVORITES ]'
favorites_header_len equ $ - favorites_header
interests_header db '[ WEEKENDS AND INTERESTS ]'
interests_header_len equ $ - interests_header

colors_text     db 'Colors: Blue and red'
colors_text_len equ $ - colors_text
perfume_text    db 'Perfume: Acqua di Gio Pour Homme'
```

Source Code Continued

```
perfume_text_len equ $ - perfume_text
music_text      db 'Music: 2000s-2010s pop/R&B and teenage-dirtbag vibes'
music_text_len  equ $ - music_text
singers_line1   db 'Singers: Justin Bieber, Eminem,'
singers_line1_len equ $ - singers_line1
singers_line2   db '         Backstreet Boys, and Oasis'
singers_line2_len equ $ - singers_line2
song_text       db "Song: Don't Look Back in Anger"
song_text_len   equ $ - song_text
food_text       db 'Food: Chocolate Cinnabon'
food_text_len   equ $ - food_text

weekend_text    db 'Weekend: Video games and basketball'
weekend_text_len equ $ - weekend_text
hobbies_text    db 'Hobbies: Video games, basketball, and chess'
hobbies_text_len equ $ - hobbies_text
tv_text         db 'TV shows: Modern Family and Shameless'
tv_text_len     equ $ - tv_text
movie_text      db 'Movie: La La Land'
movie_text_len  equ $ - movie_text
book_text       db "Book: Omniscient Reader's Viewpoint"
book_text_len   equ $ - book_text
celebrity_text db 'Celebrity: Justin Bieber'
celebrity_text_len equ $ - celebrity_text
role_model_text db 'Role model: My gramps'
role_model_text_len equ $ - role_model_text

art1          db '   _____ __   __   _____ '
art1_len        equ $ - art1
art2          db '  / ____| \ \ / /  / ____|'
art2_len        equ $ - art2
art3          db ' | |       \ V /  | (___  '
```

Source Code Continued

```
    art3_len       equ $ - art3
    art4           db ' | |        > <    \___ \ '
    art4_len       equ $ - art4
    art5           db ' | |____   / . \   ____) |'
    art5_len       equ $ - art5
    art6           db '  \_____| /_/ \_\ |_____/ '
    art6_len       equ $ - art6
    art_caption     db 'C X S'
    art_caption_len equ $ - art_caption

SECTION .bss
    current_row     resd 1
    current_col     resd 1
    target_row      resd 1
    target_col      resd 1
    message_ptr     resd 1
    message_len     resd 1

SECTION .text

_start:
    mov dword [current_row], 1
    mov dword [current_col], 1

    PRINT_AT 1,  1,  border
    PRINT_AT 2,  38, title
    PRINT_AT 3,  27, subtitle
    PRINT_AT 4,  1,  border

    PRINT_AT 6,  3,  about_header
    PRINT_AT 7,  3,  name_text
    PRINT_AT 7,  67, id_text
```

Source Code Continued

```
PRINT_AT 8,  3,  email_text
PRINT_AT 8,  67, website_text

PRINT_AT 10, 1,  border
PRINT_AT 11, 3,  firsts_header
PRINT_AT 12, 3,  achievement
PRINT_AT 13, 3,  risk_text
PRINT_AT 14, 3,  happy_line1
PRINT_AT 15, 3,  happy_line2

PRINT_AT 17, 1,  border
PRINT_AT 18, 3,  favorites_header
PRINT_AT 18, 58, interests_header
PRINT_AT 19, 3,  colors_text
PRINT_AT 19, 58, weekend_text
PRINT_AT 20, 3,  perfume_text
PRINT_AT 20, 58, hobbies_text
PRINT_AT 21, 3,  music_text
PRINT_AT 21, 58, tv_text
PRINT_AT 22, 3,  singers_line1
PRINT_AT 22, 58, movie_text
PRINT_AT 23, 3,  singers_line2
PRINT_AT 23, 58, book_text
PRINT_AT 24, 3,  song_text
PRINT_AT 24, 58, celebrity_text
PRINT_AT 25, 3,  food_text
PRINT_AT 25, 58, role_model_text

PRINT_AT 27, 1,  border
PRINT_AT 29, 38, art1
PRINT_AT 30, 38, art2
PRINT_AT 31, 38, art3
```

Source Code Continued

```
    PRINT_AT 32, 38, art4
    PRINT_AT 33, 38, art5
    PRINT_AT 34, 38, art6
    PRINT_AT 35, 49, art_caption
    PRINT_AT 37, 1,  border

    ; Finish the display with a final line break.
    mov eax, 4
    mov ebx, 1
    mov ecx, newline
    mov edx, 1
    int 0x80

    ; Exit the program successfully.
    mov eax, 1
    xor ebx, ebx
    int 0x80

; ------------------------------------------------------------------
; print_at
; Input:
;   ESI = target row
;   EDI = target column
;   ECX = address of text
;   EDX = length of text
;
; It moves forward using line breaks and spaces, then prints the
; supplied text. Calls must be arranged from top to bottom and from
; left to right when two items share the same row.
; ------------------------------------------------------------------
print_at:
    mov [target_row], esi
```

Source Code Continued

```
    mov [target_col], edi
    mov [message_ptr], ecx
    mov [message_len], edx
    pushad

    ; Move down to the requested row.
    mov ebp, [target_row]
    sub ebp, [current_row]
    cmp ebp, 0
    jle .position_column

.row_loop:
    mov eax, 4
    mov ebx, 1
    mov ecx, newline
    mov edx, 1
    int 0x80
    dec ebp
    jnz .row_loop

    mov eax, [target_row]
    mov [current_row], eax
    mov dword [current_col], 1

.position_column:
    ; Move right to the requested column.
    mov ebp, [target_col]
    sub ebp, [current_col]
    cmp ebp, 0
    jle .write_message

.column_loop:
```

Source Code Continued

```
    mov eax, 4
    mov ebx, 1
    mov ecx, one_space
    mov edx, 1
    int 0x80
    dec ebp
    jnz .column_loop

    mov eax, [target_col]
    mov [current_col], eax

.write_message:
    mov eax, 4
    mov ebx, 1
    mov ecx, [message_ptr]
    mov edx, [message_len]
    int 0x80

    mov eax, [message_len]
    add [current_col], eax
    popad
    ret
```

Final Program Output

The following screenshot shows the successfully assembled and executed slambook program in the 32-bit NASM environment.

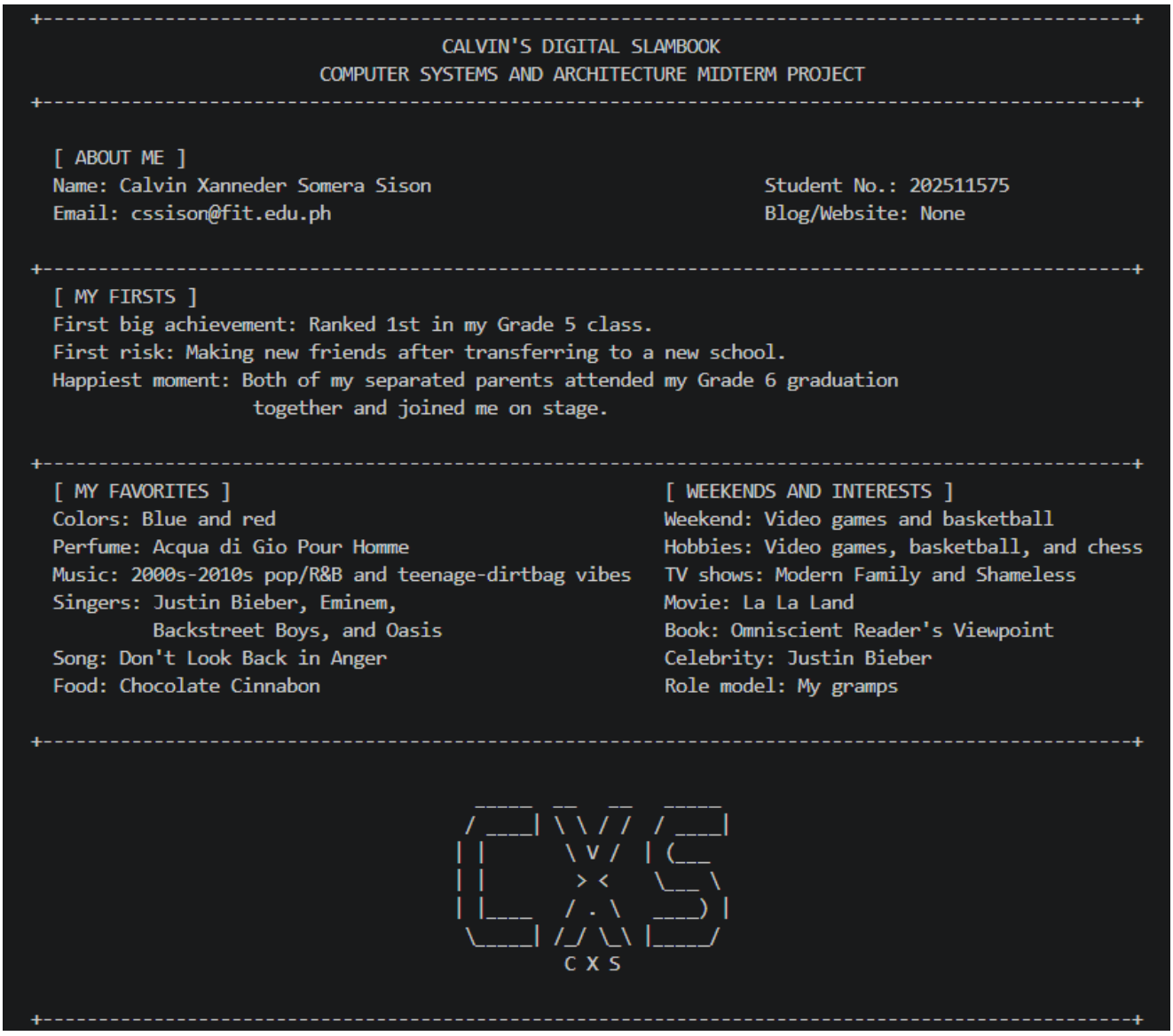


Figure 1 Final output of the digital slambook program